UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Proyecto N° 2

Juego de reflejos en el Pochoco SoC

Grupo 16:
Andrés Berríos
Víctor Parada
Sebastián Trigo

Profesor de Cátedra: Jorge Gómez M.

29 de septiembre de 2026

Índice

1. Arquitectura del juego, contador de ciclos y temporización de tres segundos 3

1.1. Arquitectura del juego . . . 3
1.2. Contador de ciclos . . . 4
1.3. Temporización de tres segundos . . . 4

2. Módulos y códigos creados para la FPGA 5

2.1. `game_top.v` . . . 5
2.2. `game.s` . . . 5
2.3. `tb_game.v` . . . 6
2.4. `assembler_v1.py` . . . 7

3. Módulos modificados 8

3.1. `pochoco_periph.v` . . . 8
3.1.1. `TENTH_CYCLES` . . . 8
3.1.2. `tenth_divider` . . . 8
3.1.3. `tenth_counter` . . . 8
3.1.4. `cycle_counter` . . . 8
3.2. `pochoco_soc.v` . . . 9
3.2.1. Carga parametrizada del firmware (`MemFile`) . . . 9
3.2.2. Enrutamiento de señales de E/S . . . 9
3.2.3. Reset de encendido . . . 9

4. Código Assembly documentado (`game.s`) 10

4.1. Inicialización del programa . . . 10
4.2. Fase inicial . . . 10
4.3. Retardo de preparación de ronda . . . 10
4.4. Selección de objetivo pseudoaleatorio . . . 11
4.5. Medición de tiempo . . . 11
4.6. Promedio de diez rondas . . . 12
4.6.1. División por diez con redondeo . . . 12
4.6.2. Saturación a dos dígitos . . . 12
4.7. Conversión a BCD . . . 13
4.8. Finalización del programa . . . 13
4.9. Mapa de memoria MMIO . . . 13

5. Assembler 15

5.1. Sintaxis aceptada 15
5.1.1. Instrucciones utilizadas por el juego 17
5.2. Funcionamiento: ensamblado en dos pasadas 18
5.3. Codificación por formato 18
5.4. Tratamiento de etiquetas 19
5.5. Tratamiento de inmediatos y validación 19

6. Simulación de resultados **20**

7. Funcionamiento en la FPGA **21**

8. Conclusiones del Proyecto **23**

1. Arquitectura del juego, contador de ciclos y temporización de tres segundos

En este apartado se incluye una breve descripción de los módulos implementados y modificados para el proyecto, haciendo una distinción entre la arquitectura general del sistema, el diseño del temporizador en hardware y los mecanismos de sincronización de tres segundos.

1.1. Arquitectura del juego

La arquitectura del juego consta de diferentes bloques funcionales y módulos de hardware interconectados, diseñados para que la FPGA funcione de la manera esperada, acorde a las especificaciones entregadas.

A continuación, se presenta la Figura 1, que muestra la arquitectura en forma de diagrama de flujo.

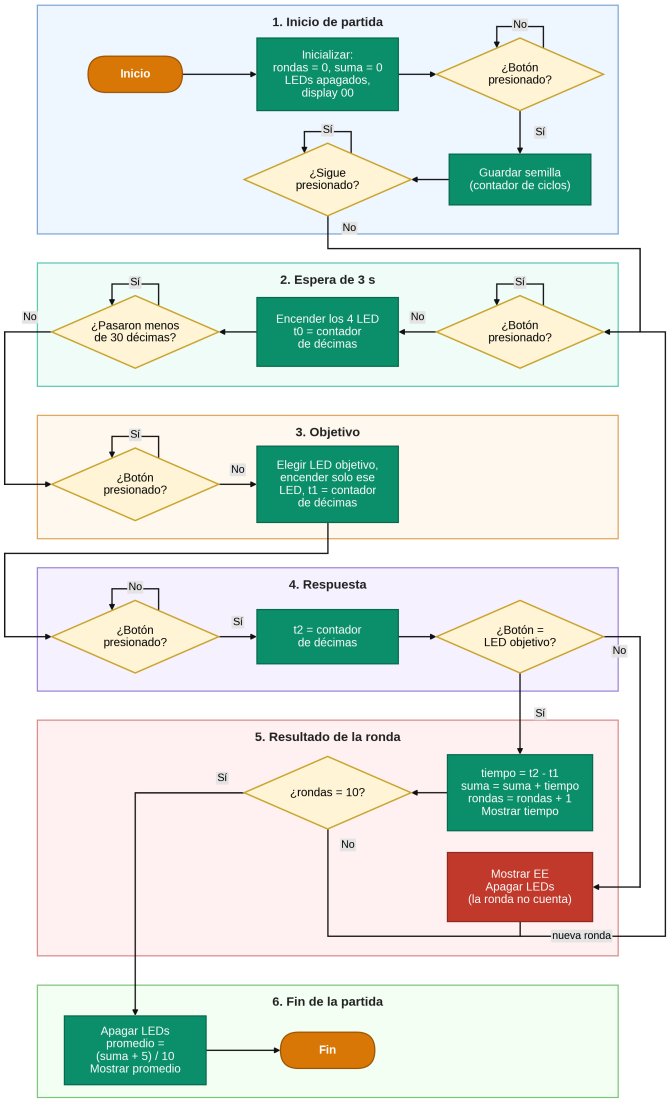


Figura 1: Diagrama de flujo del juego.

1.2. Contador de ciclos

El reloj principal de la tarjeta FPGA (`i_Clk`) opera a una frecuencia de 25 MHz. Para registrar los ciclos transcurridos, se implementó el registro `cycle_counter`, de 32 bits, dentro del módulo `pochoco_periph.v`, accesible mediante la dirección de memoria `0x8000000C`.

Este registro incrementa su valor en cada flanco de subida del reloj y completa una vuelta aproximadamente cada 171,8 s. Su reinicio solo es posible mediante un reset del sistema.

Debido a la rapidez con la que cambia su valor, el contador se utiliza como fuente de la semilla empleada para seleccionar el LED del juego, aprovechando el valor que alcanza en cada instante para introducir un componente pseudoaleatorio en la selección.

1.3. Temporización de tres segundos

Antes del inicio de cada ronda se realiza una espera de tres segundos durante la cual se encienden todos los LEDs de la placa. Para implementar esta función, se utilizaron los registros `tenth_divider` y `tenth_counter`, incorporados en el módulo `pochoco_periph.v`.

El registro `tenth_divider`, de 22 bits, funciona como *prescaler* contando los ciclos del reloj principal del sistema hasta alcanzar el valor definido por el parámetro `TENTH_CYCLES`, para posteriormente reiniciar su cuenta y emitir un pulso de habilitación. Con el objetivo de manejar una escala de décimas de segundo al implementar el juego, se le asigna al parámetro `TENTH_CYCLES` un valor de 2,500,000 ciclos, lo equivalente a una décima parte de la frecuencia del reloj principal (25 MHz), lo cual fija el periodo del pulso emitido por `tenth_divider` en 0,1 s. Durante las etapas de simulación, el valor de `TENTH_CYCLES` se reduce a 100 ciclos con el propósito de acelerar las pruebas del sistema.

Los pulsos emitidos por el *prescaler* son almacenados en el registro `tenth_counter`, de 16 bits, cuyo valor representa, en décimas de segundo, el tiempo transcurrido desde que se inició el programa.

Para implementar la temporización de tres segundos, el programa realiza una comparación entre el valor inicial registrado por `tenth_counter` al inicio de la ronda y su valor actual, durante la cual se mantienen encendidos todos los LEDs de la placa, hasta que la diferencia alcanza un umbral de 30 décimas de segundo. Una vez que el programa registra dicho valor, se transiciona automáticamente hacia la siguiente fase del juego.

2. Módulos y códigos creados para la FPGA

En este apartado se describen los módulos implementados para el proyecto, explicando el propósito de cada uno, su función dentro de la arquitectura del sistema y su contribución al funcionamiento general del juego de reflejos.

2.1. `game_top.v`

El módulo `game_top` es el módulo de nivel superior del diseño. Es el que se sintetiza y se programa en la FPGA Lattice iCE40 HX1K de la Go Board. Su propósito es servir de interfaz entre el SoC `pochoco_soc` y los pines físicos de la placa, y seleccionar la imagen de memoria que el SoC carga al iniciar, que en este proyecto es el programa del juego, `sw/game.hex`. Su estructura se resume en tres puntos: la declaración de puertos, la instanciación del SoC y el manejo de la UART:

- **Declaración de puertos.** Expone los puertos que se asocian a los pines de la placa en el archivo de restricciones `goboard.pcf`: el reloj `i_Clk`, los cuatro LEDs (`o_LED[3:0]`), los cuatro botones (`i_Switch[3:0]`), los siete segmentos de cada uno de los dos displays (`o_Segment1_A`, entre otros).
- **Instanciación del SoC.** Instancia `pochoco_soc` con el nombre `u_pochoco` y conecta uno a uno los puertos de `game_top` con los del SoC. Al instanciarlo, redefine el parámetro `MemFile` con el valor `"sw/game.hex"`, de modo que la memoria RAM unificada del SoC se inicialice con el programa del juego en lugar de con el programa por defecto del repositorio base.
- **Manejo de la UART.** El juego no usa comunicación serial, por lo que `game_top` fija la salida `o_UART_TX` en nivel alto (`1'b1`), que es el nivel de reposo de una línea UART, y no utiliza la entrada `i_UART_RX`. Ambos puertos se mantienen declarados porque sus pines aparecen en `goboard.pcf`.

2.2. `game.s`

El archivo `game.s` contiene el código en lenguaje ensamblador (RV32E) del juego de reflejos. Es el programa que ejecuta el procesador Espino Core y concentra toda la lógica del juego: la secuencia de cada ronda, la selección pseudoaleatoria del LED objetivo, la medición del tiempo de respuesta, el manejo de los errores, el conteo de las diez rondas correctas y el cálculo del promedio. Este archivo es traducido por el assembler propio del grupo a `game.hex`, que `game_top` carga en la memoria RAM del SoC al iniciar.

Este módulo es la parte del sistema que define el comportamiento observable del juego. El hardware aporta los registros de E/S y los contadores, mientras que `game.s` ordena su uso para cumplir la secuencia de cada ronda, que consiste en la fase inicial de tres segundos, la selección pseudoaleatoria del objetivo, la medición del tiempo, el manejo de errores, las diez rondas correctas y el promedio final.

2.3. `tb_game.v`

El archivo `tb_game.v` es un *testbench* de simulación que verifica el funcionamiento completo del juego sin necesidad de programar la FPGA. No forma parte del hardware sintetizado: su propósito es comprobar, antes de la demostración en la placa, que el programa `game.hex` produce la secuencia esperada de LEDs, el manejo de errores y la finalización tras diez rondas correctas. Para ello se ejecuta en un simulador Verilog con una escala de tiempo de 1 ns/1 ps.

Dentro de la arquitectura del sistema, el testbench cumple el papel de la placa y del jugador. Instancia directamente el módulo `pochoco_soc` con el programa `sw/game.hex`, genera un reloj con periodo de 40 ns (25 MHz), y conecta los botones a un registro que el propio testbench controla, mientras observa los LEDs y los segmentos de los displays. Las entradas SPI se dejan inactivas, porque el juego no las usa. Para acelerar la simulación, el parámetro `TenthCycles`, que define cuántos ciclos de reloj forman una décima de segundo, se fija en 100 en lugar de los 2 500 000 que se usan en la FPGA real. Además, el testbench lee mediante una referencia jerárquica el registro interno de los displays (`dut.u_periph.digit_q`), lo que permite comprobar lo que el programa escribe en ellos.

En cuanto a su contribución al juego, el testbench reproduce una partida completa. Primero realiza la pulsación inicial; luego, en cada ronda, espera a que se enciendan los cuatro LEDs y después un único LED objetivo, y responde con el botón correspondiente tras un tiempo de reacción simulado de 250 ciclos de reloj. En la tercera ronda introduce deliberadamente una pulsación incorrecta y espera que el display muestre `EE`, lo que verifica el reinicio de la ronda actual sin perder las rondas correctas acumuladas. Al completar diez respuestas correctas, deja un tiempo para el cálculo del promedio y entrega un resumen con el número de rondas correctas, si se probó el error, el estado final de los LEDs y el valor del display. La prueba se considera exitosa si hubo diez rondas correctas, se probó el error y los LEDs quedaron apagados. El valor del promedio solo se imprime, por lo que debe revisarse manualmente. Finalmente, un temporizador de 400 000 ciclos termina la simulación si el juego se detiene.

2.4. `assembler_v1.py`

El archivo `assembler_v1.py` es el ensamblador desarrollado en el lenguaje Python. Su propósito es traducir el programa en lenguaje ensamblador `game.s` a `game.hex`, un archivo de texto con una palabra de 32 bits en hexadecimal por línea, que es el formato que el SoC carga en su memoria al iniciar. Se ejecuta desde la línea de comandos con `python3 assembler_v1.py entrada.s salida.hex`. Reconoce las instrucciones del conjunto base RV32I (aritméticas y lógicas, cargas y almacenamientos, saltos condicionales, `jal`, `jalr`, `lui` y `auipc`) y restringe los registros a `x0`–`x15`, que son los que tiene el Espino Core. Usa solo los módulos `sys` y `re` de la biblioteca estándar y no invoca ningún ensamblador RISC-V externo, como exige el enunciado.

Su funcionamiento es el de un ensamblador de dos pasadas. En la primera, recorre el archivo, elimina los comentarios (`#` y `//`), ignora directivas como `.section` y `.global`, y asigna a cada instrucción su dirección, que aumenta de 4 en 4 bytes. Cada etiqueta se registra en una tabla con la dirección en que aparece. En la segunda pasada, una tabla indica, para cada mnemónico, su formato (R, I, S, B, U o J) y los campos fijos `opcode`, `funct3` y `funct7`. Con esos datos y los operandos, una función por formato arma los 32 bits mediante desplazamientos y operaciones OR. Los inmediatos pueden escribirse en decimal, hexadecimal o binario. En los saltos `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu` y en `jal`, el operando es una etiqueta y el inmediato se calcula como la diferencia entre la dirección de la etiqueta y la de la instrucción actual, lo que permite saltos hacia adelante y hacia atrás. Además, el ensamblador valida la entrada y detiene el proceso con un mensaje de error en casos como una instrucción desconocida, un registro inválido, una etiqueta duplicada, un inmediato que no cabe en su campo o un desplazamiento impar en un salto.

Dentro del sistema, el ensamblador conecta el software con el hardware: transforma la lógica del juego escrita en `game.s` en el contenido de `game.hex`, que `game_top` selecciona mediante el parámetro `MemFile` y que el procesador ejecuta. Esto hace posible el flujo de trabajo pedido, en que cualquier modificación del juego, como cambiar una constante, se aplica regenerando `game.hex` con este programa y volviendo a sintetizar y programar la FPGA. Su codificación se verificó comparando su salida con los archivos `.hex` de los programas de ejemplo del repositorio base (`blink`, `7seg` y `buttons_leds`), y resultó idéntica en los tres casos.

3. Módulos modificados

En esta sección se describen las modificaciones realizadas sobre la arquitectura base del proyecto (`pochoco_soc`). Los cambios se introdujeron para integrar la lógica del juego de reflejos, la medición de tiempos de respuesta mediante un temporizador en hardware y la interfaz de control requerida para la FPGA.

3.1. `pochoco_periph.v`

El módulo `pochoco_periph.v` representaba los registros estáticos para la lectura y escritura de los distintos periféricos que componen la FPGA. Para dar soporte a las mediciones realizadas durante el juego de reflejos, se reestructuró la lógica del periférico, incorporando temporizadores síncronos.

3.1.1. `TENTH_CYCLES`

`TENTH_CYCLES` es un parámetro sin dirección mapeada en memoria dentro del módulo, el cual funciona como un factor de división de frecuencia utilizado para escalar el tiempo. Su valor por defecto es de 2,500,000 ciclos, lo cual corresponde a una décima parte de la frecuencia del reloj principal del sistema, escalando el tiempo a décimas de segundo. Su valor puede reducirse durante la simulación con el fin de acelerar las pruebas del sistema.

3.1.2. `tenth_divider`

El reloj principal de la FPGA funciona con una frecuencia de 25 MHz. Para lograr una medición en décimas de segundo, se implementó el contador `tenth_divider`, de 22 bits, que funciona como *prescaler* y genera un pulso de habilitación cada 2 500 000 ciclos de reloj (definidos por el parámetro `TENTH_CYCLES`), representando cada intervalo una décima de segundo.

3.1.3. `tenth_counter`

Es un registro de 16 bits que se incrementa con cada pulso de `tenth_divider`. Representa las décimas de segundo transcurridas desde que se energizó la tarjeta.

3.1.4. `cycle_counter`

Es un registro de 32 bits, que aumenta su cuenta por cada ciclo del reloj principal.

3.2. `pochoco_soc.v`

Este módulo corresponde al núcleo del sistema en chip (*SoC*), encargado de interconectar el procesador, la memoria RAM unificada y el bloque de periféricos. Este módulo fue modificado con el fin de integrar la nueva lógica interna requerida para el juego de reflejos y, además, para que el inicio de la FPGA no tuviera problemas.

3.2.1. Carga parametrizada del firmware (`MemFile`)

Se reconfiguró la memoria interna del SoC incluyendo el parámetro `MemFile`, con el valor `"sw/game.hex"`. Este parámetro canaliza la ruta hacia la directiva `$readmemh` dentro del bloque de memoria RAM, permitiendo cargar de manera dinámica el ejecutable, que gestiona la secuencia de las 10 rondas, la verificación de respuestas, el cálculo del promedio y el ciclo de reinicio.

3.2.2. Enrutamiento de señales de E/S

Se expuso el bus de 4 bits de la interfaz del SoC y se conectó a la entrada `btn_i[3:0]` de `pochoco_periph.v`, para asegurar que las lecturas de la CPU representen de manera sincronizada las pulsaciones de los botones por parte del jugador. A su vez, se mapearon las salidas de los periféricos `leds_o`, `seg1_o` y `seg2_o` como salidas formales del SoC, permitiendo enviar los vectores decodificados hacia los controladores encargados de encender los LED objetivo y representar los dígitos de tiempo en los displays de siete segmentos.

3.2.3. Reset de encendido

Durante las pruebas realizadas de manera física a la FPGA, al realizar la carga del programa del juego, este no siempre iniciaba. Al encender o reprogramar la FPGA, el procesador necesita un *reset* inicial que lo mantenga detenido mientras el reloj y el resto del circuito se estabilizan. El contador original de 4 bits que mantenía este reset duraba 15 ciclos de reloj, lo que en tiempo real equivale a menos de un microsegundo. Esta duración es demasiado corta y provocaba que el procesador arrancara bien o mal de manera aleatoria e inconsistente.

Para corregir el comportamiento errático de inicio, se reemplazó el contador de 4 bits por uno de 16 bits. Con el contador nuevo, el estado de *reset* dura aproximadamente *2,6* milisegundos (65 535 ciclos a 25 MHz), tiempo suficiente para que el sistema se estabilice antes de iniciar el programa, que en este caso es el juego.

4. Código Assembly documentado (`game.s`)

El archivo `game.s` contiene el programa principal del juego escrito en lenguaje ensamblador RISC-V (RV32E). En esta sección se detallan las direcciones de memoria de los registros y periféricos utilizadas en la FPGA, además de documentar el funcionamiento general del código y las distintas fases que componen su flujo de ejecución.

4.1. Inicialización del programa

Al comenzar la ejecución del archivo `game.s`, el programa configura las referencias de memoria y las variables globales necesarias para el flujo del juego, consolidadas en la Tabla 1. En el registro `x2` se carga la dirección base de los periféricos mapeados en memoria mediante la instrucción `lui x2, 0x80000`, permitiendo acceder a los botones, LEDs y displays.

A continuación, los registros `x3` (rondas correctas) y `x4` (suma de tiempos) se inicializan en cero utilizando la instrucción `addi`. Para garantizar un estado inicial limpio en la interfaz física de la FPGA, se escriben ceros en las direcciones de los LEDs (`4(x2)`) y los displays de siete segmentos (`0(x2)`).

4.2. Fase inicial

Una vez configurados los registros, el sistema entra en la etiqueta `wait_start_press` a la espera de que el usuario inicie la partida. El programa realiza un monitoreo activo (*polling*) sobre la dirección de los pulsadores `8(x2)`, aplicando la máscara `andi x1, x1, 15` para aislar el estado de los cuatro botones.

Al detectar la pulsación de cualquier botón, el programa lee el valor de `cycle_counter` desde `12(x2)` y lo guarda en `x14`, que constituye la semilla inicial de la selección pseudoaleatoria. Posterior a ello, el programa entra en la etiqueta `wait_start_release`, ejecutando un bucle de espera hasta verificar la liberación total de los botones para evitar que la pulsación de inicio se registre erróneamente como una respuesta a una ronda del juego.

4.3. Retardo de preparación de ronda

Cada ronda inicia en la etiqueta `round_start`, donde el programa comprueba que todos los botones se encuentren liberados. Luego, enciende los cuatro LEDs escribiendo la constante `0xF` en la dirección `4(x2)`, y lee el valor instantáneo de `tenth_counter` desde `16(x2)`, guardándolo en `x6` como la marca de tiempo inicial t_0. Además, carga en el registro `x12` el valor 30, correspondiente a los tres segundos de espera expresados en décimas.

A continuación, el programa entra en el bucle `wait_three_seconds`, donde lee repetidamente la dirección `16(x2)` hacia el registro `x7` y calcula el tiempo transcurrido aplicando la siguiente resta:

$$\texttt{x7} \leftarrow \texttt{x7} - \texttt{x6} \tag{1}$$

El bucle se repite mientras el valor de `x7` sea menor que `x12`, utilizando la instrucción `bltu`, es decir, hasta que transcurran las 30 décimas de segundo. Se calcula esta diferencia en lugar de comparar con un valor absoluto, debido a que `tenth_counter` se incrementa de forma continua desde el *reset* del sistema y no se reinicia en cada ronda.

Una vez completados los tres segundos, el programa valida la liberación de los botones en la etiqueta `wait_before_target` antes de pasar a la selección del objetivo. De esta forma, una pulsación realizada durante la espera no se interpreta como respuesta a la ronda: si el botón se suelta antes de que terminen los tres segundos, se ignora, y si sigue presionado, el programa espera a que se libere antes de seleccionar el objetivo.

4.4. Selección de objetivo pseudoaleatorio

Para seleccionar cuál de los cuatro LEDs servirá como objetivo en la ronda de forma pseudoaleatoria, el programa genera y actualiza la semilla en el registro `x14`. Para ello, a partir de la semilla capturada en la pulsación inicial, el programa lee en cada ronda el valor instantáneo de `cycle_counter` desde la dirección `12(x2)` hacia el registro auxiliar `x8` y aplica la siguiente serie de operaciones *XOR*:

```
lw   x8, 12(x2)
xor  x14, x14, x8
xor  x14, x14, x4
xor  x14, x14, x3
xori x14, x14, 1445
andi x8, x14, 3
```

Posterior a ello, se aplica una máscara binaria al resultado obtenido mediante `andi x8, x14, 3`, restringiendo el resultado a los valores $0, 1, 2$ o 3. De acuerdo con el resultado obtenido, se selecciona la máscara del LED individual correspondiente ($1, 2, 4$ u 8) en el registro `x5`.

Finalmente, el flujo salta a la etiqueta `target_ready`, donde se escribe la máscara cargada sobre la dirección de los LEDs, para mostrar únicamente el objetivo que ha de seleccionar el jugador.

4.5. Medición de tiempo

Una vez encendido el LED objetivo en la etiqueta `target_ready`, el programa guarda inmediatamente la marca de tiempo inicial t_1 mediante la lectura `lw x6, 16(x2)` desde `tenth_counter`. Esta captura se realiza en el mismo instante lógico en que inicia el estímulo visual para garantizar la precisión de la medición.

A continuación, el programa ingresa al bucle `wait_button`, monitoreando constantemente la entrada de los botones desde la dirección `8(x2)`. Al registrar una pulsación, se guarda de inmediato el instante de respuesta t_2 en el registro `x7` realizando una nueva lectura desde `16(x2)`. Para obtener el tiempo de respuesta final (Δt), se calcula la diferencia entre ambas marcas de tiempo mediante la instrucción:

```
sub x9, x7, x6
```
(2)

El resultado almacenado en `x9` representa el tiempo de reacción expresado en décimas de segundo, lo que otorga una resolución de medición de 0,1 s.

Posteriormente, en la subrutina de verificación, se valida si el botón presionado coincide con el LED objetivo utilizando la instrucción `bne x1, x5, wrong_button`. Si los valores no coinciden, el flujo se desvía a la etiqueta `wrong_button` para manejar el error; de lo contrario, el programa prosigue con el procesamiento del tiempo obtenido.

Listing 1: Captura de marcas de tiempo y bucle de lectura de pulsación.

```
target_ready:
    sw x5, 4(x2)            # Encender LED objetivo
    lw x6, 16(x2)           # t1: Marca de tiempo inicial (tenth_counter)

wait_button:
    lw x1, 8(x2)            # Leer estado de los botones
    beq x1, x0, wait_button # Esperar hasta que se presione un boton
    lw x7, 16(x2)           # t2: Marca de tiempo final al pulsar
    sub x9, x7, x6          # Tiempo de respuesta = t2 - t1 (resolucion 0.1s)
```

Evaluación de respuesta e incremento de tiempo

Durante el juego, cada vez que se obtiene una respuesta correcta, el tiempo de reacción t_i (en décimas de segundo) se suma al registro `x4`, y el contador de rondas correctas `x3` se incrementa en uno. Una pulsación incorrecta no modifica ninguno de los dos registros, por lo que los tiempos de las rondas reiniciadas no afectan el promedio. Al terminar la décima ronda, el registro `x4` contiene la suma de los diez tiempos.

4.6. Promedio de diez rondas

Al completar la décima respuesta correcta, el programa calcula y muestra el tiempo promedio de respuesta en la misma unidad utilizada durante el juego, que es de décimas de segundo. El cálculo se realiza en la etiqueta `game_finished` de `game.s` y se divide en dos etapas: la acumulación de los tiempos y la división por diez del total acumulado.

4.6.1. División por diez con redondeo

El promedio buscado se obtiene al dividir por diez el registro `x4`. La división se implementa mediante restas sucesivas. Para que el resultado se redondee al entero más cercano en lugar de truncarse, se suma 5 a la suma total S antes de dividir.

En el programa, el valor $S + 5$ se copia al registro `x11` y el cociente se acumula en `x9`, que parte en cero. El lazo `average_loop` compara `x11` con 10 mediante `slti`. Mientras `x11` sea mayor o igual a 10, resta 10 a `x11` y suma 1 a `x9`. Cuando `x11` es menor que 10, el lazo termina y `x9` contiene el promedio en décimas de segundo. Como siempre se promedian exactamente diez rondas, el divisor 10 es constante.

4.6.2. Saturación a dos dígitos

Los displays disponibles solo permiten mostrar dos dígitos decimales. Si el promedio resulta mayor o igual a 100 décimas, el programa lo limita a 99 (equivalente a 9,9 segundos) en la etiqueta `average_done`, para evitar mostrar un valor incorrecto.

4.7. Conversión a BCD

Cada display de siete segmentos muestra un dígito, y el periférico interpreta el byte escrito como dos grupos de cuatro bits: los cuatro bits altos corresponden al display de las decenas y los cuatro bits bajos al de las unidades. Por lo tanto, el promedio debe convertirse a formato BCD (decimal codificado en binario). El lazo `average_bcd_loop` resta 10 repetidamente a `x9` y cuenta cuántas veces lo hace en `x10`. Al terminar, `x10` contiene las decenas y `x9` las unidades.

Luego, el byte final se arma como:

$$\texttt{x11} = 16 \cdot \texttt{x10} + \texttt{x9} \tag{3}$$

Como no se pueden usar desplazamientos, la multiplicación por 16 se realiza sumando el registro consigo mismo cuatro veces consecutivas, lo que equivale a duplicar el valor cuatro veces ($2^4 = 16$).

4.8. Finalización del programa

Una vez desplegado el tiempo promedio final en los displays de siete segmentos, el flujo de ejecución del programa ingresa a un bucle infinito en la etiqueta `finished` mediante la instrucción de salto incondicional `jal x0, finished`.

Esta instrucción mantiene al procesador en un lazo infinito sin modificar los periféricos, por lo que la interfaz física queda congelada y el valor del promedio permanece en pantalla de manera indefinida.

4.9. Mapa de memoria MMIO

En esta sección se presentan las direcciones de memoria mapeadas (MMIO) y la especificación técnica de los periféricos integrados en la FPGA para el funcionamiento de `game.s`.

La Tabla 1 detalla la ubicación en memoria de cada periférico junto con su función específica dentro de la lógica del juego. Por su parte, la Tabla 2 especifica las características de hardware a nivel de ancho de bus, tipo de acceso y la procedencia de cada registro dentro del módulo Verilog `pochoco_periph.v`.

Dirección	Registro	Acceso	Uso en el juego
`0x80000000`	Displays de 7 segmentos	Escritura (`sw`)	Mostrar el tiempo de respuesta, el promedio final y el código de error `0xEE`.
`0x80000004`	LEDs	Escritura (`sw`)	Encender los cuatro LEDs durante la espera y luego solo el LED objetivo.
`0x80000008`	Botones	Lectura (`lw`)	Detectar el inicio de la partida, las pulsaciones y su liberación.
`0x8000000C`	`cycle_counter`	Lectura (`lw`)	Obtener la semilla de la selección pseudoaleatoria.
`0x80000010`	`tenth_counter`	Lectura (`lw`)	Medir el retardo de 3 segundos y los tiempos de respuesta en décimas de segundo.

Tabla 1: Registros de periféricos utilizados por `game.s`.
Fuente: Elaboración propia.

Dirección	Registro	Acceso	Ancho	Contenido	Origen
`0x80000000`	Displays	Escritura	8 bits	Dos dígitos BCD: bits [7:4] display decenas y bits [3:0] display unidades.	Base
`0x80000004`	LEDs	Escritura	4 bits	Un bit por LED individual.	Base
`0x80000008`	Botones	Lectura	4 bits	Un bit por botón presionado.	Base
`0x8000000C`	`cycle_counter`	Lectura	32 bits	Ciclos de reloj transcurridos desde el *reset*.	Nuevo
`0x80000010`	`tenth_counter`	Lectura	16 bits	Décimas de segundo transcurridas desde el *reset*.	Nuevo

Tabla 2: Mapa de registros implementados en `pochoco_periph.v`.

Fuente: Elaboración propia.

5. Assembler

Esta sección describe el assembler del proyecto: su sintaxis, la codificación de las instrucciones, el tratamiento de etiquetas e inmediatos, la validación de errores y las pruebas realizadas.

El assembler fue desarrollado en su totalidad en Python, usando exclusivamente la biblioteca estándar (`sys` y `re`) y sin ningún assembler externo. Toda la traducción, desde el texto hasta los bits de cada instrucción, se realiza dentro del programa.

Recibe como entrada un archivo en assembly (en el proyecto, `game.s`) y genera `game.hex`, con una palabra de 32 bits por línea escrita en hexadecimal. Este es el formato que lee la directiva `$readmemh` de `pochoco_ram.v`. Se ejecuta, desde la carpeta `pochoco_soc`, con:

```
python3 assembler_v1.py sw/game.s sw/game.hex
```

El `Makefile` ejecuta este comando como primer paso, regenerando `game.hex` cuando `game.s` o el assembler son más recientes que él. De esta forma, el flujo desde el código fuente en assembly hasta el *bitstream* y la programación de la FPGA queda automatizada.

5.1. Sintaxis aceptada

El conjunto de reglas que rigen el assembler, el cual corresponde a un subconjunto de la sintaxis de RISC-V para RV32E, se presenta en la Tabla 3. En este subconjunto no se admiten pseudoinstrucciones (`nop`, `li`, `mv`, `j`, `ret`), directivas de datos (`.word`, `.equ`) ni nombres ABI de registros, y solo se aceptan los registros `x0` hasta `x15`.

Elemento	**Regla**	**Ejemplo**
Comentarios	Todo lo que sigue a `#` o `//` hasta el final de la línea se ignora.	`# espera`
Etiquetas	Identificador seguido de `:`. Puede ir sola en la línea o antes de una instrucción, y comenzar con letra, `_` o `.`.	`round_start:`
Directivas	`.section`, `.global`, `.globl`, `.text` y `.data` se reconocen y se ignoran, para que `game.s` también sea válido con el toolchain GNU.	`.global _start`
Mnemónicos	Se convierten a minúscula, por lo que no distinguen mayúsculas.	`ADDI` o `addi`
Operandos	Separados por comas.	`addi x1, x0, 15`
Registros	Solo `x0` a `x15` (RV32E). No se aceptan nombres ABI.	`x14`
Inmediatos	Decimal, hexadecimal (`0x`) o binario (`0b`), con signo opcional.	`15`, `-8`, `0xF`, `0b1111`
Acceso a memoria	Se escribe `inmediato(registro)`.	`4(x2)`
Etiqueta como operando	Solo en saltos condicionales (formato B) y en `jal`.	`bne x1, x0, round_start`

Tabla 3: Sintaxis aceptada por el assembler.
Fuente: Elaboración Propia

La Tabla 4 resume las instrucciones que reconoce el assembler, agrupadas por formato, junto con el opcode que utiliza cada grupo.

Formato	**Instrucciones**	**Opcode**
R	`add, sub, sll, slt, sltu, xor, srl, sra, or, and`	`0x33`
I (aritmética)	`addi, slti, sltiu, xori, ori, andi`	`0x13`
I (desplazamiento)	`slli, srli, srai`	`0x13`
I (carga)	`lb, lh, lw, lbu, lhu`	`0x03`
I (salto)	`jalr`	`0x67`
S	`sb, sh, sw`	`0x23`
B	`beq, bne, blt, bge, bltu, bgeu`	`0x63`
U	`lui`	`0x37`
U	`auipc`	`0x17`
J	`jal`	`0x6f`

Tabla 4: Instrucciones soportadas por el assembler.
Fuente: Elaboración Propia

La instrucción `jalr` se escribe con la sintaxis de acceso a memoria, es decir, `jalr rd, imm(rs1)`.

Los desplazamientos (`sll`, `srl`, `sra` y sus versiones con inmediato) permanecen deshabilitados para ahorrar LUTs. Por esta razón, el juego utiliza una serie de cuatro sumas consecutivas para la conversión a BCD.

5.1.1. Instrucciones utilizadas por el juego

El programa `game.s` utiliza 14 instrucciones distintas, indicadas en la Tabla 5.

Instrucción	**Formato**	**Uso en el juego**
`lui`	U	Cargar la dirección base de los periféricos (`0x80000000`).
`addi`	I	Inicializar registros y constantes, incrementar el contador de rondas y restar 10 en las conversiones.
`add`	R	Acumular los tiempos de respuesta y multiplicar por 16 mediante sumas.
`sub`	R	Calcular el tiempo transcurrido como diferencia entre dos lecturas del contador de décimas.
`xor`	R	Mezclar la semilla con el contador de ciclos, la suma de tiempos y el número de ronda.
`xori`	I	Aplicar una constante a la semilla pseudoaleatoria.
`andi`	I	Aislar los 4 bits de los botones y extraer los 2 bits que eligen el LED objetivo.
`slti`	I	Comparar un valor con 10 o con 100 en las conversiones.
`lw`	I	Leer botones, contador de ciclos y contador de décimas.
`sw`	S	Escribir en los LEDs y en los displays.
`beq`	B	Esperar pulsaciones y seleccionar el LED objetivo.
`bne`	B	Verificar el botón y repetir bucles de espera.
`bltu`	B	Mantener la espera de 3 segundos mientras no se alcancen las 30 décimas.
`jal`	J	Saltos incondicionales (con `x0` como destino, para no guardar dirección de retorno).

Tabla 5: Instrucciones utilizadas por `game.s`.
Fuente: Elaboración Propia

5.2. Funcionamiento: ensamblado en dos pasadas

El assembler recorre el archivo dos veces, lo que permite usar etiquetas definidas más adelante en el código (saltos hacia adelante).

1. **Primera pasada.** Se lee cada línea, se elimina el comentario y se separa la etiqueta, si existe. Cada etiqueta se registra en un diccionario con la dirección de la siguiente instrucción, y el contador de dirección aumenta en 4 por cada instrucción. Las líneas vacías y las directivas no consumen dirección. Si una etiqueta se define dos veces, el assembler termina con un error.
2. **Segunda pasada.** Con todas las etiquetas ya resueltas, se recorre la lista de instrucciones. Para cada una se busca el mnemónico en la tabla de instrucciones, que entrega su formato, opcode, `funct3` y `funct7`. Luego se leen los operandos y se construye la palabra de 32 bits con el codificador del formato correspondiente.

Las palabras resultantes se escriben en orden en `game.hex`. Como la RAM del SoC comienza en la dirección 0 y el core arranca en `BootAddr = 0`, la primera instrucción del archivo es la primera que se ejecuta.

5.3. Codificación por formato

Cada formato distribuye los 32 bits de la instrucción en campos distintos, como muestra la Tabla 6.

	31:25	**24:20**	**19:15**	**14:12**	**11:7**	**6:0**
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
I (shamt)	funct7	shamt	rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

Tabla 6: Campos de cada formato de instrucción.
Fuente: Elaboración Propia

Cada formato tiene su propia función de codificación en el assembler (`encode_r`, `encode_i`, `encode_shamt`, `encode_s`, `encode_b`, `encode_u` y `encode_j`). Todas desplazan cada campo a su posición y los combinan con operaciones OR.

Los formatos **B** y **J** almacenan el inmediato con los bits desordenados. El bit 0 no se guarda, porque las instrucciones siempre están alineadas y el desplazamiento es par, y el bit de signo siempre queda en la posición 31. Esta distribución, definida por la especificación de RISC-V, hace que la mayoría de los bits del inmediato coincidan en posición con los de otros formatos, lo que simplifica el decodificador de hardware (`espino_decoder.v` reconstruye el inmediato con la misma ordenación).

5.4. Tratamiento de etiquetas

Cuando una instrucción de salto usa una etiqueta como operando, el assembler calcula un desplazamiento relativo a la dirección de la propia instrucción:

$$\text{desplazamiento} = \text{dirección de la etiqueta} - \text{dirección de la instrucción}$$

Este valor es el que se codifica en el campo de inmediato. En el hardware, el core calcula el destino como `pc_id + imm_target` (módulo `espino_id_stage.v`), por lo que ambos extremos usan la misma referencia. Un desplazamiento negativo corresponde a un salto hacia atrás, como ocurre en todos los bucles de espera del juego.

Las etiquetas solo se resuelven en los formatos B y J. En ambos, el operando también puede ser un número, que se interpreta directamente como el desplazamiento relativo.

5.5. Tratamiento de inmediatos y validación

Los inmediatos se convierten a entero con `int(valor, 0)`, que reconoce automáticamente los prefijos `0x` y `0b` y los números negativos. Antes de codificar, el assembler verifica los siguientes puntos y, si alguno falla, termina con un mensaje de error y no genera el archivo:

- **Mnemónico desconocido:** la instrucción no está en la tabla.
- **Registro inválido:** cualquier nombre distinto de `x0` a `x15`, lo que garantiza compatibilidad con RV32E.
- **Inmediato inválido:** el texto no es un número ni una etiqueta definida.
- **Rango de inmediatos:** 12 bits con signo para los formatos I y S (de -2048 a 2047), 13 bits con signo para los saltos condicionales (de -4096 a 4094) y 21 bits con signo para `jal`.
- **Cantidad de desplazamiento (`shamt`):** debe estar entre 0 y 31.
- **Alineación:** los desplazamientos de los saltos deben ser pares.
- **Etiqueta duplicada:** una misma etiqueta no puede definirse dos veces.
- **Formato de acceso a memoria:** los operandos de carga y almacenamiento deben tener la forma `inmediato(registro)`.

Para `lui` y `auipc`, el inmediato se trunca a 20 bits, de modo que `lui x2, 0x80000` produce la dirección base de los periféricos.

6. Simulación de resultados

Para confirmar que los módulos creados para este proyecto funcionan correctamente, se ha realizado una simulación con *iverilog* y *vvp*, que representa lo que sería el flujo normal del juego. En este caso, la simulación tiene once rondas en total, ya que una de ellas falla a propósito. Además, se muestran los mensajes del estado final del juego, que indican que este funciona completamente en simulación. Se muestra la imagen de la simulación mencionada en la Figura 2.

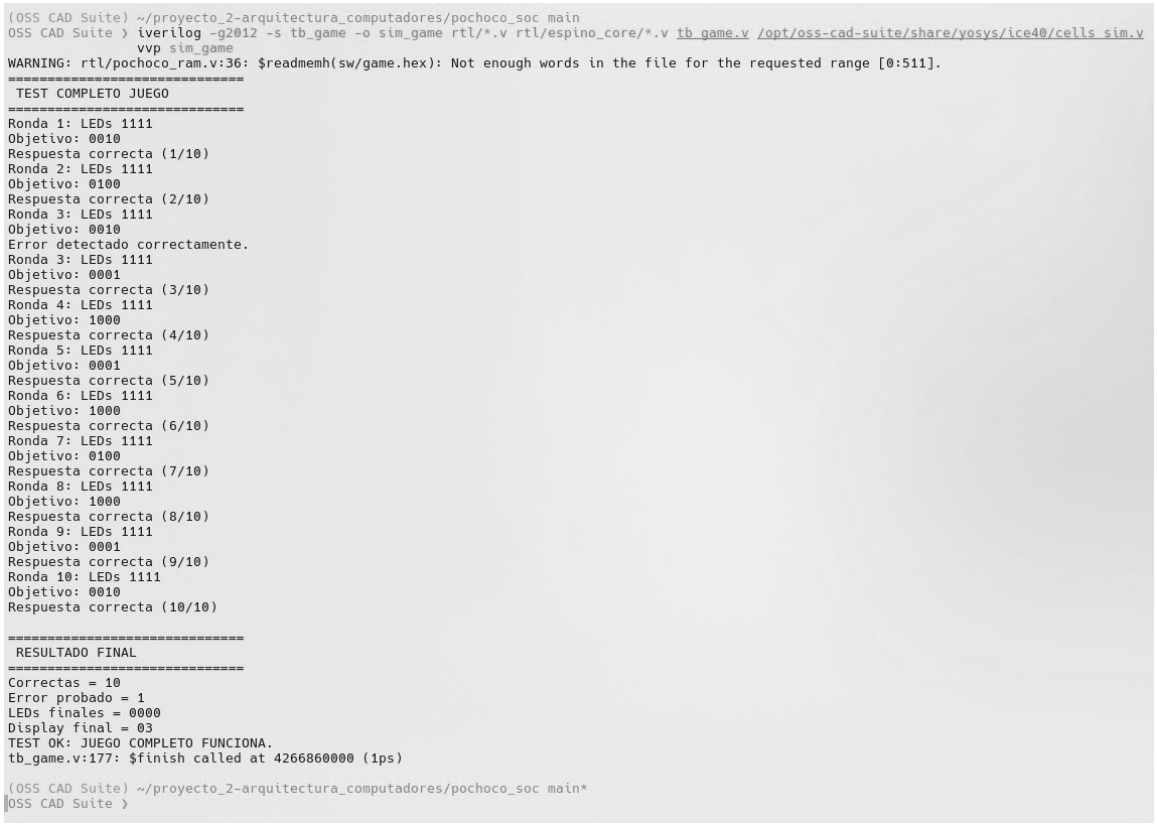


Figura 2: Simulación del Testbench (Imagen en escala de grises)
Fuente: Elaboración Propia

7. Funcionamiento en la FPGA

Con la simulación realizada, se procede a cargar el juego en la FPGA real, usando el flujo indicado en el archivo *README.md* del repositorio del proyecto en GitHub.

Mediante las pruebas reales se confirma que la FPGA funciona correctamente y que no se encuentra ningún tipo de problema, ya que los que se fueron presentando a medida que avanzaba el proyecto fueron resueltos al momento de ser encontrados, por lo que se considera que la versión final del proyecto cumple con todos los requerimientos pedidos para el juego. Además, se presentan las Figuras 3, 4, 5 y 6, que muestran la FPGA en el estado inicial (previo a iniciar el juego), en la segunda ronda, en una ronda donde se presiona el botón erróneo y en el estado final (después de diez rondas correctas).



Figura 3: FPGA: Estado Inicial

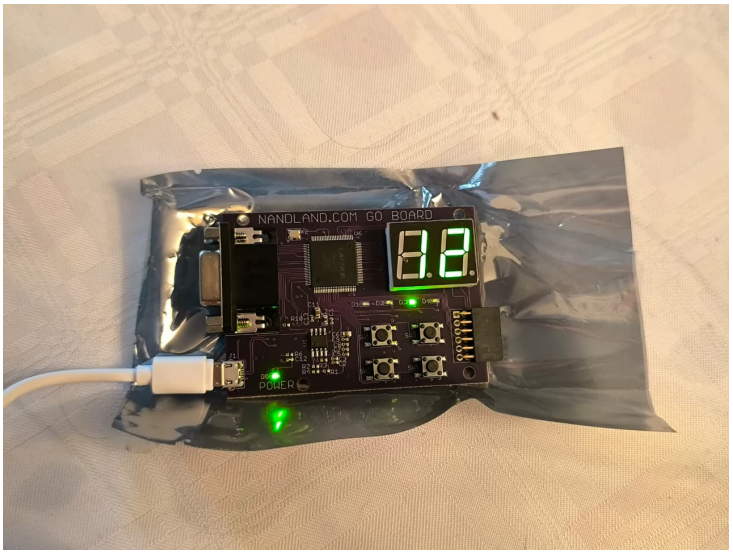


Figura 4: FPGA: Segunda Ronda



Figura 5: FPGA: Ronda con Botón Equivocado



Figura 6: FPGA: Estado Final

La razón por la que se escogen estas cuatro imágenes es mostrar que la FPGA funciona correctamente, además de que son los estados más representativos del juego de reflejos.

8. Conclusiones del Proyecto

El desarrollo de este proyecto permitió conocer en la práctica el funcionamiento de la FPGA Lattice iCE40 HX1K y de la Go Board, desde la síntesis del diseño hasta su programación y prueba en la placa. También permitió comprender cómo se organiza un sistema completo: un procesador (el Espino Core), una memoria unificada y periféricos accesibles mediante un mapa de memoria, y cómo un programa interactúa con el hardware a través de LEDs, botones y displays de siete segmentos.

Otro aprendizaje importante fue trabajar directamente en lenguaje ensamblador. Escribir el juego en `game.s` obligó a pensar en términos de registros, direcciones de memoria y saltos, y a resolver operaciones que en lenguajes de alto nivel son inmediatas. Como las instrucciones de desplazamiento están deshabilitadas en la ALU del procesador, la división por diez y la conversión a BCD se implementaron con restas y sumas sucesivas. Además, desarrollar el assembler propio en Python permitió entender cómo cada instrucción se codifica en una palabra de 32 bits, cómo se resuelven las etiquetas mediante dos pasadas y cómo se genera el archivo `game.hex` que carga el SoC. La verificación de su salida contra los archivos `.hex` de los programas de ejemplo del repositorio base resultó idéntica en los tres casos.

Se cumplió el objetivo de desarrollar un juego de reflejos que funciona en la FPGA. El sistema enciende los cuatro LEDs durante tres segundos al inicio de cada ronda, selecciona un LED objetivo de forma pseudoaleatoria a partir de una semilla tomada del contador de ciclos, mide el tiempo de respuesta en décimas de segundo, maneja las pulsaciones incorrectas reiniciando la ronda sin perder las rondas correctas acumuladas, y, tras diez respuestas correctas, calcula y muestra el tiempo promedio. Para lograrlo se agregó al periférico un contador de ciclos y un contador de décimas de segundo, accesibles desde el programa. El testbench `tb_game.v` permitió además comprobar el funcionamiento completo del juego en simulación antes de usar la placa.

Durante el desarrollo surgieron problemas que fueron resueltos modificando el código base. El más relevante fue un arranque intermitente de la FPGA: en algunas cargas el procesador quedaba detenido, con los displays en “00” y sin responder a los botones. Mediante marcadores en los displays y repetidas cargas del mismo bitstream se determinó que la falla ocurría en el arranque y no en la lógica del juego. La causa era que el reset de encendido, un contador de 4 bits, duraba solo 15 ciclos de reloj, un tiempo insuficiente para que el sistema se estabilizara. Al ampliar ese contador a 16 bits, con una duración cercana a 2,6 milisegundos, el problema desapareció y el juego arrancó de forma consistente, incluso al desconectar y volver a conectar la placa. Este episodio mostró la importancia de documentar y verificar las modificaciones al RTL.

En conclusión, el proyecto fue desarrollado de manera exitosa, ya que se aplicaron correctamente los conceptos estudiados en el curso: arquitectura de un procesador y su SoC, entrada y salida mapeada en memoria, programación en ensamblador, traducción de assembly a código máquina, diseño digital en Verilog y verificación mediante simulación y pruebas en hardware real. El resultado es un sistema completo y funcional, en el que cada componente, desde el assembler hasta la placa, fue comprendido y puesto a prueba por el grupo.